Components, contracts, and the data model

HyperspaceVault · Architecture · View 2 of 3 — Logical

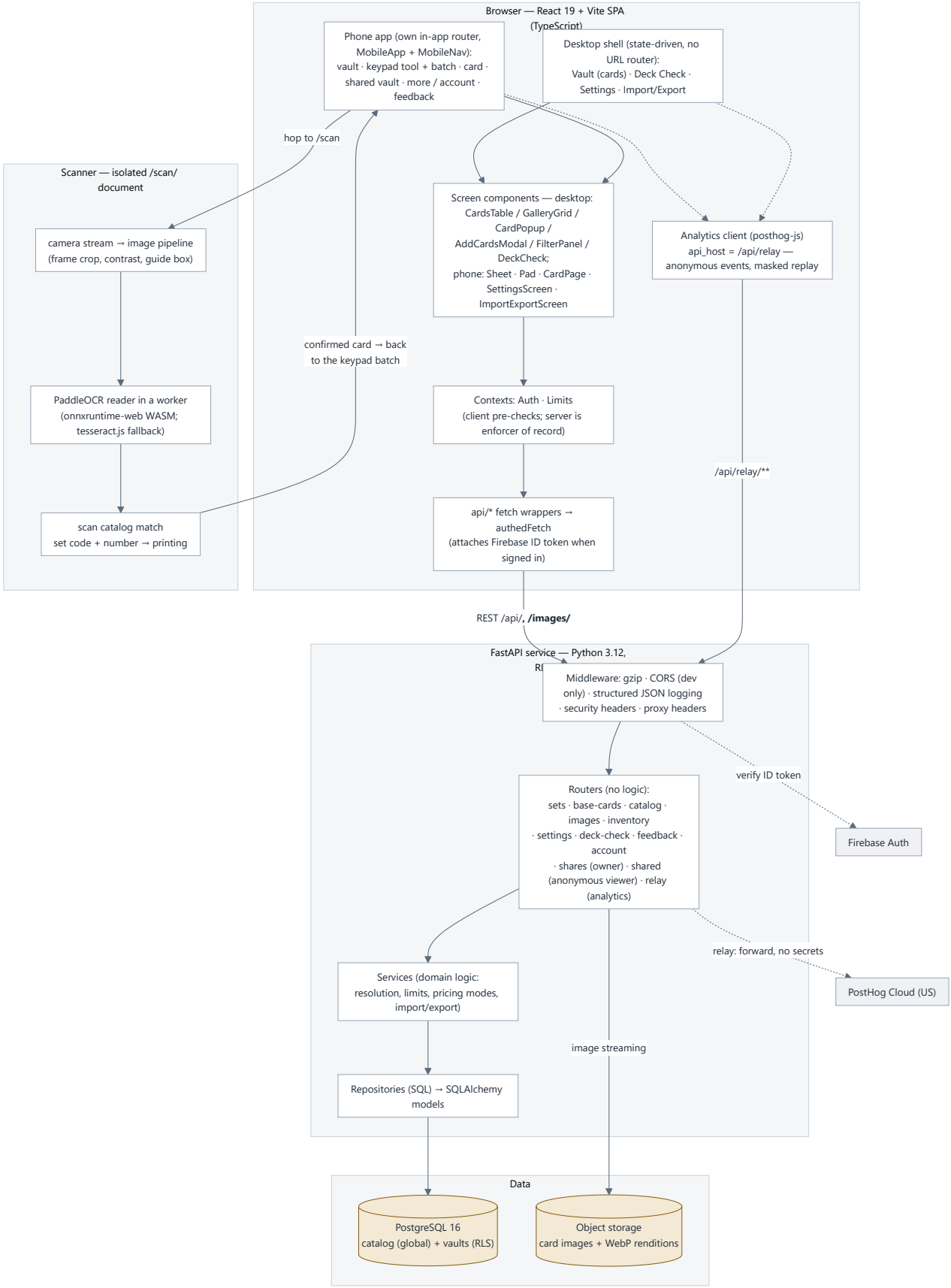
A React single-page app — rendered as a desktop shell or a phone app from the same bundle, plus a second, isolated document for the camera scanner — talks to one FastAPI service over REST; PostgreSQL holds both the shared catalog and per-tenant vaults, with tenant isolation enforced by the database itself. Around that core sit the ingestion pipelines that keep catalog, images, and prices current.

Prepared 2026-07-28 · Updated 2026-10-18 — as of release v2.0.1 (prod `c65cf87`) with the v2.1 candidate on dev (main `d3118c5`) · Companion views: Conceptual (domain & actors) · Physical (deployment & delivery)

Component layering

Both sides of the wire enforce a strict internal layering. The frontend keeps all server talk in thin fetch wrappers over one auth-aware client; the backend keeps routers free of logic, pushing it down through services to repositories.

Logical components



Which client you get is decided once at load by one shared predicate: a viewport under 768 px, or a touch-first device whose screen's short side is at most 500 px, renders the phone app (`MobileApp`) instead of the desktop shell; a phone turned on its side shows a "turn your phone upright" screen over the mounted app. The

desktop shell is still state-driven with no URL router (it parses only `/shared/{token}`); the phone app has its own in-app router because its screens are navigated back and forth with the browser's history.

The scanner is a separate document on purpose. On-device OCR needs `SharedArrayBuffer`, which browsers grant only to cross-origin-isolated pages. Rather than isolate the whole app (and lose embedded third-party content), the scanner lives in its own `/scan/` document that Hosting serves with COOP `same-origin` + COEP `require-corp` and `Permissions-Policy: camera=(self)`; the reader bundle, its WASM runtime and the OCR models are vendored under `/vendor/scan/` (fetched on first use, cached immutable), and the worker bundles under `/scan-workers/`, both served with `Cross-Origin-Resource-Policy: same-origin`. The phone app hops to `/scan` and back; nothing about the main document's headers changes.

API surface

Route group	Purpose	Access
`/api/sets`	Set list and detail	`PUBLIC · CDN-CACHED`
`/api/base-cards`	Catalog list & card detail, price history; detail merges in your quantities when signed in	`PUBLIC (OPTIONAL AUTH)`
`/images/cards/*`	Card image streaming (3 renditions)	`PUBLIC · 1-YEAR IMMUTABLE CACHE`
`/api/catalog/reference.csv`	Resolution-key reference for importers	`PUBLIC`
`/api/inventory`	Quantities, increment / decrement / adjust, import / export	`AUTH · MUTATIONS + IMPORT/EXPORT NEED VERIFIED EMAIL`
`/api/deck-check`	Ephemeral three-scope deck diff with pricing	`AUTH`
`/api/settings/limits`	Keep-limit matrix, hard/soft cap mode	`AUTH`
`/api/feedback`	Feedback → GitHub issue (best-effort)	`OPTIONAL AUTH · RATE-LIMITED`
`/api/account`	Idempotent full-tenant delete	`AUTH + 5-MIN RECENT RE-AUTH`
`/api/shares`	Create / rename / rotate / revoke view-only collection shares	`AUTH`
`/api/shared/{token}`	Anonymous read-only vault view — the token *is* the credential	`PUBLIC · TOKEN-SCOPED · RATE-LIMITED · NEVER LOGGED`
`/api/relay/*`	First-party reverse proxy for PostHog (event ingestion, config, replay assets) — forwards to `us.i.posthog.com` / `us-assets.i.posthog.com`	`PUBLIC · UNAUTHENTICATED BY DESIGN · HOLDS NO SECRETS · TOUCHES NO APP STATE`

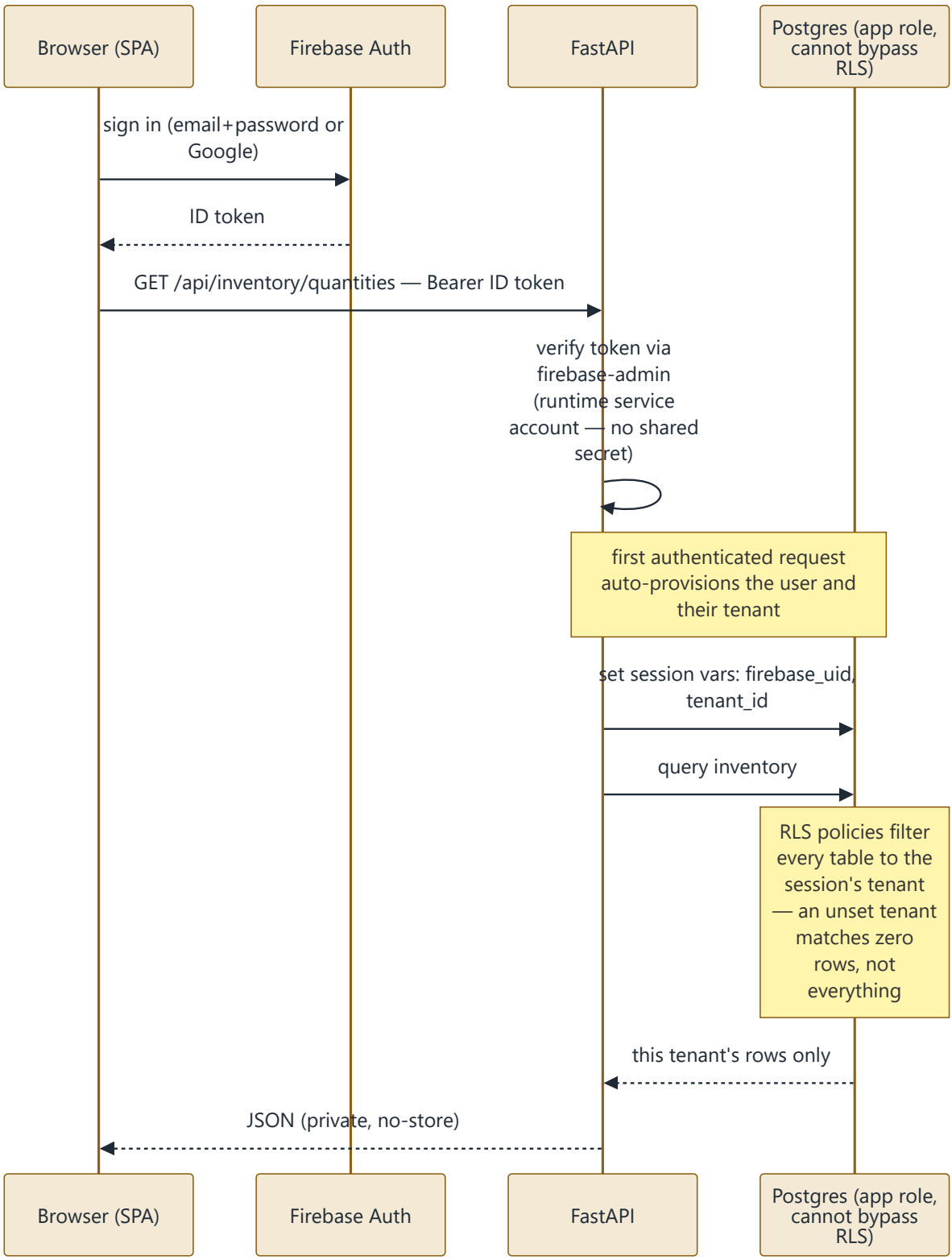
Auth requirements are expressed as a ladder of FastAPI dependencies — each endpoint declares exactly the trust level it needs: no database at all (the relay, image streaming, health), tenant-less catalog access, token-scoped anonymous share reads (a 256-bit secret link resolves, via RLS itself, to one tenant's read-only view), optional auth, required auth (which auto-provisions the user and tenant on first contact), verified email, and finally recent re-authentication for account deletion. Since 2026-10-18 the ladder is **machine-checked**: a

backend test introspects every route's dependencies and fails CI unless the route-registry table in the Platform Spec (§1.3) records the same session dependency and gates — a new or changed route cannot ship undocumented.

Identity and tenant isolation

The load-bearing decision: **isolation is enforced by Postgres row-level security, not by application code**. Every request runs as a database role that cannot bypass RLS; the request's identity is injected as session variables, and policies do the filtering. A forgotten `WHERE tenant_id = …` in application SQL returns zero rows instead of another collector's vault.

Authenticated request — token to tenant-scoped rows



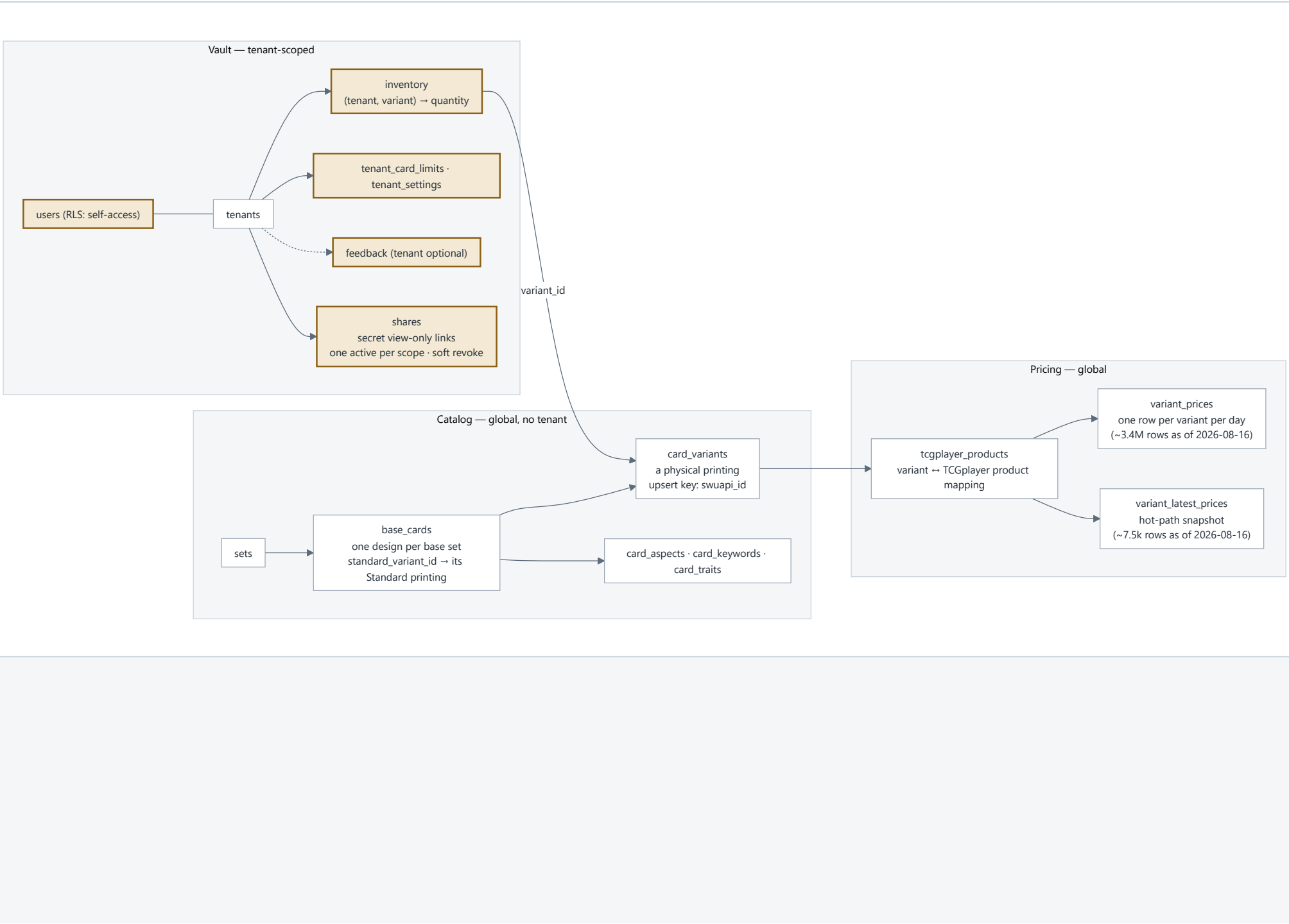
- **Two database roles by design:** a privileged role (may bypass RLS) used only by migrations and ingestion pipelines; an application role (cannot bypass RLS) used by every request. The session is pinned to one pooled connection for the request's life so the identity variables can't leak across requests.
- **Fail-closed policies:** the tenant-matching expression was hardened so a missing or cleared tenant matches *no* rows — there is no default-tenant fallback.

- **One user = one tenant**, auto-provisioned; the tenants table itself is scoped by server-derived filters (deletes only), everything tenant-owned is under forced RLS.
- **Sharing rides the same rails (v1.4):** a share link's secret token resolves — through an RLS policy, not application code — to its owner's tenant, and the anonymous viewer then reads *identical rows through identical queries* as the owner would; there is no share-specific read path to audit. No write endpoint exists on the viewer surface, revocation is immediate, and invalid and revoked tokens are deliberately indistinguishable (both 404). Viewer responses are cacheable for 30 s, privately.

Data model

Three clusters: the global catalog, global pricing keyed to catalog variants, and the tenant-scoped vault tables. Amber marks tables under **forced row-level security**.

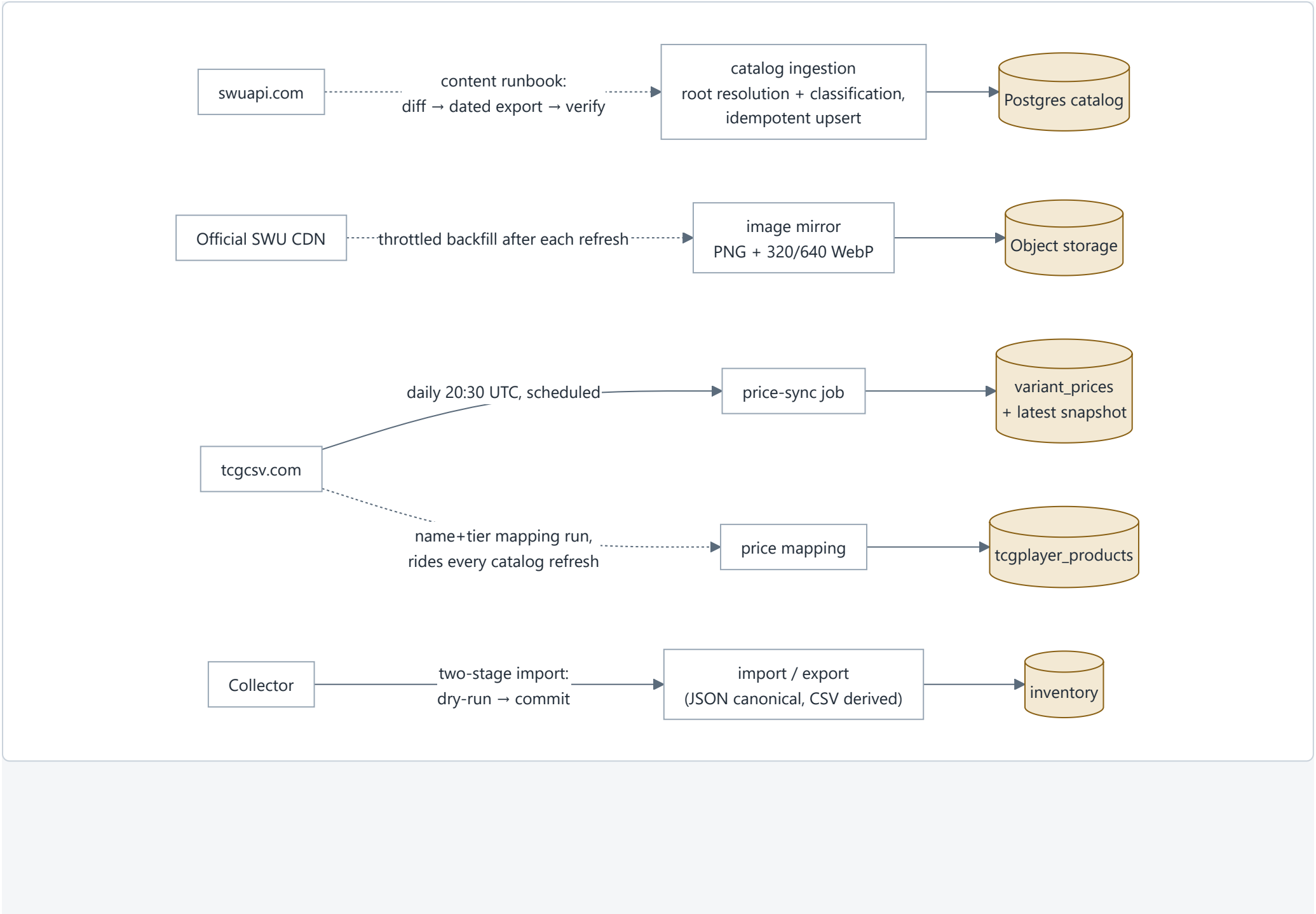
Core tables and relationships (medium detail)


Vault — tenant-scoped
inventory
(tenant, variant) → quantity
tenant_card_limits ·
tenant_settings
users (RLS: self-access)
tenants
feedback (tenant optional)
shares
secret view-only links
one active per scope · soft revoke
variant_id
Catalog — global, no tenant
card_variants
a physical printing
upsert key: swuapi_id
sets
base_cards
one design per base set
standard_variant_id → its
Standard printing
card_aspects · card_keywords ·
card_traits
Pricing — global
tcgplayer_products
variant ↔ TCGplayer product
mapping
variant_prices
one row per variant per day
(~3.4M rows as of 2026-08-16)
variant_latest_prices
hot-path snapshot
(~7.5k rows as of 2026-08-16)

- **History vs. hot path:** daily prices append to a history table (866 days of depth for charts as of 2026-08-16, growing by one day per sync); a separate latest-price snapshot keeps list-view reads flat regardless of history size.
- **Idempotent catalog writes:** ingestion upserts on the source system's stable ID, so a re-run of the same export is a no-op — additive, single-transaction, safe to retry.
- **Variant classification is derived, not stored:** the raw source label is kept verbatim; finish and channel are computed on read by the same classifier ingestion uses — one source of truth, no denormalization drift.

Data pipelines — automated vs. operator-gated

Ingestion flows (dashed = operator-run)


swuapi.com
content runbook:
diff → dated export → verify
catalog ingestion
root resolution + classification,
idempotent upsert
Postgres catalog
Official SWU CDN
throttled backfill after each refresh
image mirror
PNG + 320/640 WebP
Object storage
daily 20:30 UTC, scheduled
price-sync job
variant_prices
+ latest snapshot
tcgcsv.com
name+tier mapping run,
rides every catalog refresh
price mapping
tcgplayer_products
Collector
two-stage import:
dry-run → commit
import / export
(JSON canonical, CSV derived)
inventory

Flow	Cadence	Trigger
Price sync (daily builds)	Daily, 20:30 UTC	`AUTOMATED` — scheduler
Catalog refresh + image backfill	Per new set / content change	`OPERATOR RUNBOOK` — verified end-to-end, prod gated
Price mapping	With every catalog refresh	`OPERATOR` — new printings never price without it
Price history backfill	One-time (complete)	`OPERATOR` — resumable archive walk, 3.4M rows (2026-07)
Inventory import / export	On demand	`COLLECTOR` — dry-run preview, then one-transaction commit (desktop and phone)

> **Honest caveat baked into the design:** catalog *currency detection* is manual today — there is no automated "new set appeared" watcher. That is a documented operating decision (runbook-driven content), not an accident; automating detection is tracked backlog work.

Prepared 2026-07-28 · Updated 2026-10-18 — as of release v2.0.1 (prod `c65cf87`) with the v2.1 candidate on dev (main `d3118c5`), from the as-built system; companion views: Conceptual · Physical